PINPOINT

[AN AI-POWERED GEOGRAPHIC LOCATION IDENTIFICATION SYSTEM USING MULTI-STAGE VISUAL REASONING]

Arjun Bhatnagar

Abstract

Identifying geographic locations from images without the use of meta-data is difficult as it requires simultaneous processing of multiple visual identifiers. This is something most humans struggle with and existing automated systems handle it poorly, often relying purely on landmark recognition or unreliable single-model prompting. Pinpoint was built to overcome this problem. It is a 13-stage AI pipeline combining OpenCV and Gemini 2.5 Flash to identify a country, region and state from a single street view screenshot. Pinpoint was built using a grounded reasoning approach, comparing live imagery against real reference examples rather than relying on model knowledge alone. It consists of a reference database of 1840+ images across 125 countries. It achieved 8% country level accuracy and has wider applications in fields of OSINT (Open-Source Intelligence), border security and aerial imagery analysis.

INTRODUCTION AND AIM:

Problem statement:

Identifying a geographic location from an image or two is a challenging task which requires one to simultaneously process dozens of visual signals. This is something most humans struggle with and most existing automated systems handle poorly.

Current approaches can be broadly categorized into 2: pure model prompting- fast but unreliable and pure database matching – can't generalize and is too rigid. Thus, neither alone is sufficient. To overcome this, Pinpoint was built to combine both approaches into a single coherent system.

What is Pinpoint:

Pinpoint is an autonomous 13 stage AI pipeline that takes a street view screenshot as an input and produces a country, region and state-level prediction as an output. It primarily consists of three core components- OpenCV for image quality filtering, Gemini 2.5 Flash for visual reasoning and a large reference database of 1840+ images across 125 countries. Pinpoint operates semi-autonomously. After a user opens a street view image and presses G, the pipeline runs independently through all 13 stages and outputs a final prediction for the location of the image.

Aim and objectives:

The aim of this project was to design and build an AI program capable of identifying geographic location from Street View imagery with measurable accuracy at country, region, and state level.

The various objectives set for this project are:

1. Research and define a set of visual “metas” that reliably differentiate geographic locations across countries.
2. Build a reference image database using websites such as geohints.com and covering as many meta categories and countries as possible.
3. Design a multi stage pipeline capable of selecting high-quality screenshots, performing grounded reasoning, retrieving images from the reference database and finally, producing a final prediction.
4. Implement state and region-level reasoning for at least 10 major countries using hand-crafted geographic knowledge bases.
5. Build an accuracy tracking system to systematically test performance in every round.
6. Identify, analyze and fix failure cases to improve the pipeline over time

Wider Purpose:

Beyond a general geo-detection system, Pinpoint has various applications in the real world. This includes:

1. OSINT (Open-Source Intelligence): Journalists and intelligence analysts need to routinely verify the locations of social media images and conflict footage [6]. Pinpoint demonstrates that this time-intensive process can be semi-automated.
2. Surveillance and border security: Border security and law enforcement frequently need to identify the location of images and footage without the use of GPS metadata. This is currently done manually by trained analysts, but any real-world deployment in this context would require careful consideration of privacy implications
3. Aerial and Satellite imagery: The meta category approach used by Pinpoint can be applied to aerial images for disaster response, humanitarian work and land use monitoring.

Background Research

GeoGuessr is a popular geography game released in 2013 [13]. In this, players are dropped into random google street view locations and aim to identify where they are by observing their surroundings. Rather than completely focusing on one thing such as language, players must simultaneously process various visual clues such as road markings, vegetation, sun-direction, driving side and more in order to pinpoint an accurate guess. Professional players have developed a concept of visual "metas"- visual categories which remain largely consistent across countries. These are systematic identifiers that distinguish one country from another. Pinpoint aims to teach an AI system to think like an expert GeoGuessr player-analyze and identify all possible meta categories which references to known examples for each country.

What are meta categories:

As previously stated, "metas" are visual characteristics that are largely constant all throughout a country. They carry immense geographical information and in many cases are the key differences between two seemingly identical countries. The 13 meta categories used by Pinpoint are: bollards, utility poles, traffic lights, terrain, road signs, license plates, language/script, flora, architecture, road surface, street markings, street names, signposts.

Each category is largely independent of the other. For example, even if the language of a sign cannot be identified, utility poles or license plates can still be helpful towards identifying the country. The combination of various meta categories has even more significance. Even if a country shares a language/script with more countries, combining this with knowledge of bollards or utility poles can help narrow down the country dramatically. The primary resource used to study meta categories was geohints.com, a database created by Twitch users MouseWithBeer and PV_93 [8] containing country specific meta examples across all major meta categories.

Why use AI Vision:

Traditional computer vision detection models (edge detection, color histograms, template matching) are currently too weak [1]. Although it can identify that a bollard is present, it cannot reason what country that particular bollard style belongs to. Large Multimodal Models like Gemini 2.5 Flash are more suited towards this objective as they can process images quickly and reason about them in natural language. Therefore, they are able to identify country specific

metas much more accurately. This model was selected as it is capable of processing images and reasoning about them in natural language, with the likelihood that Google's training data includes substantial Street View imagery. If a regular AI model was given a street view picture and prompted to guess the country, it may hallucinate confidently [4]. This motivated the reference database approach.

Reference Database Approach:

Instead of asking Gemini to rely purely on training knowledge, which would result in large amounts of hallucination, it uses a reference database containing real examples of what each country looks like across each meta-category. Then, Gemini compares the live screenshot against these reference images to identify the most likely country. This is known as grounded reasoning [2]; the model's output relies on real evidence rather than statistical patterns in training data. This is similar to how a doctor diagnoses patients- rather than diagnosing from memory alone, they compare the case against various references, test results and known patterns. Pinpoint does the same for geographic identification. This leads to a stark reduction in hallucination [5] because the model now compares images rather than generating a guess from scratch. To maximize accuracy, a database consisting of more than 1840 images across 125 different commonly encountered countries was used to provide meaningful coverage.

Why use a multi-stage pipeline rather than a single API call?

If the program only used a single call consisting of 1-2 screenshots, although you would receive an answer, it would be much less accurate because there is no reasoning chain, confidence calibration and more importantly, no way to catch errors.

A multi-stage pipeline externalizes the reasoning [3]- each stage only does one specific job, completes it and passes it onto the next stage. This means that you are able to inspect, debug and improve individual stages without having to spend hours finding one specific error or end up rebuilding everything. For example, when the program identified Eswatini instead of South Africa, inspecting each stage's JSON output made it clear that the error occurred in the grounded reasoner rather than other stages like the first-pass reasoning. This would have been extremely difficult to identify with a single API (Application-Programming Interface) call. Research has shown that breaking down complex reasoning into intermediate steps significantly improves model performance [3], and production AI systems commonly rely on chains of specialized steps rather than a single call. Pinpoint's 13-stage pipeline follows this same principle, ensuring that every prediction is the result of a transparent, step-by-step reasoning process rather than a single unchecked guess

Considered Approaches and Plan

Throughout the process of building Pinpoint, various approaches were considered. Each approach was evaluated based on accuracy, speed and debuggability before the final multi-stage pipeline design was chosen.

1. **Single model call:** This was the simplest possible approach. It involved taking a screenshot, sending it directly to Gemini and prompting it to guess the country. Although this approach was the fastest to build and required no pipeline, it had various flaws. It produced confident but wrong answers with no reasoning chain, no way to analyze the certainty of the model and had a high hallucination risk as there was no reference material to compare screenshots to.
2. **Pure computer vision:** This approach involved using OpenCV to extract visual features such as edge patterns, color distributions and texture analysis and match them against a reference database using a similarity metric. This was fast and had no rate limiting since it didn't involve any API calls. However, alone, this was not enough. Although computer vision could detect the presence of an object like a bollard, it could not accurately predict what country it belonged to as geographic reasoning required semantic understanding, something only a LLM (Large-Language Model) could provide.

3. **OCR-first pipeline:** This approach involved using OCR (Optical Character Recognition) to extract text from screenshots first -road signs, shop names, license plates- and use that text as the primary geographic identifier. This was considered as text is highly informative. For example, a speed limit sign in km/h vs mph can narrow countries down dramatically. Initially this was implemented in the pipeline. However, this was removed as it significantly increased the amount of time per run, adding almost 1 minute per run with almost zero incremental value as Gemini 2.5 Flash already reads and reasons about text. Thus, a separate OCR stage was completely unnecessary

4. **Multi-stage pipeline with grounded reasoning:** After evaluating all three approaches, the final design combined the strengths of multiple approaches rather than solely focusing on a single one. It combines Gemini's semantic reasoning (addressing the lack of understanding in pure computer vision) grounded against a real reference database (prevents single-model hallucination) and uses computer vision software such as OpenCV to filter and rank images before being processed. This entire process is divided into 13 stages thus, errors can be inspected and debugged without rebuilding the entire system.

Time planning:

Phase	Tasks	Approximate Time
Research	Meta-category research, pipeline architecture design and overall project setup	2 Weeks
Core Pipeline	Smart capture system, first-pass reasoning and basic pipeline structure	2 Weeks
Reference Database	AI tool to scrape images for reference database, populated /Country_Data with 1840+ images.	1 Week
Advanced reasoning	Retrieval engine, grounded reasoning and confidence verifier	2 Weeks
Sub-location system	Sub-location reasoning, knowledge base JSONs for 15 countries	1.5 Weeks
Testing and refinement	Accuracy tracking, bug fixing, pipeline optimization	2 Weeks
		70+ hours total

Materials and Resources

1. Software Tools:

- **Python** [11]**:** primary programming language throughout the project. All 13 pipeline modules were written in Python.
- **Gemini 2.5 Flash API** [7]**:** Core AI reasoning engine. It was used across various pipeline stages including first-pass reasoning, image selection, grounded comparison and sub-location reasoning.
- **OpenCV** [9]**:** Used for scoring image quality, detecting duplicate and visually similar frames and detecting geographic evidence density in the smart capture stage.
- **Pyautogui** [10]**:** screen capture and mouse/keyboard automation for interacting with google street view and GeoGuessr
- **Standard Python libraries** [11]**:** json, os, pathlib, requests, re, time – used for file handling, JSON parsing, HTTP requests and pipeline orchestration.

2. Data Sources:

- **geohints.com** [8]**:** Source of a large majority of reference images. It is a community built GeoGuessr resource documenting visual metas across a large number of countries. A custom web scraper (geohints_scraper.py) was written to navigate the site's meta category pages and categorically download all reference images for the database.

3. AI assistance:

- **Claude AI & Claude Code (Anthropic)** [12] **:** Used as a technical architecture advisor throughout the project. It was primarily used for improving on the pipeline architecture, debugging specific modules and discussing design decisions such as the confidence calibration logic. All implementation such as running and testing code and making final decisions carried out independently.

4. The full codebase is publicly available at https://github.com/ArjunnBhatnagar/pinpoint.

A website for the project has also been made to explain the functioning clearly. https://arjunnbhatnagar.github.io/pinpoint

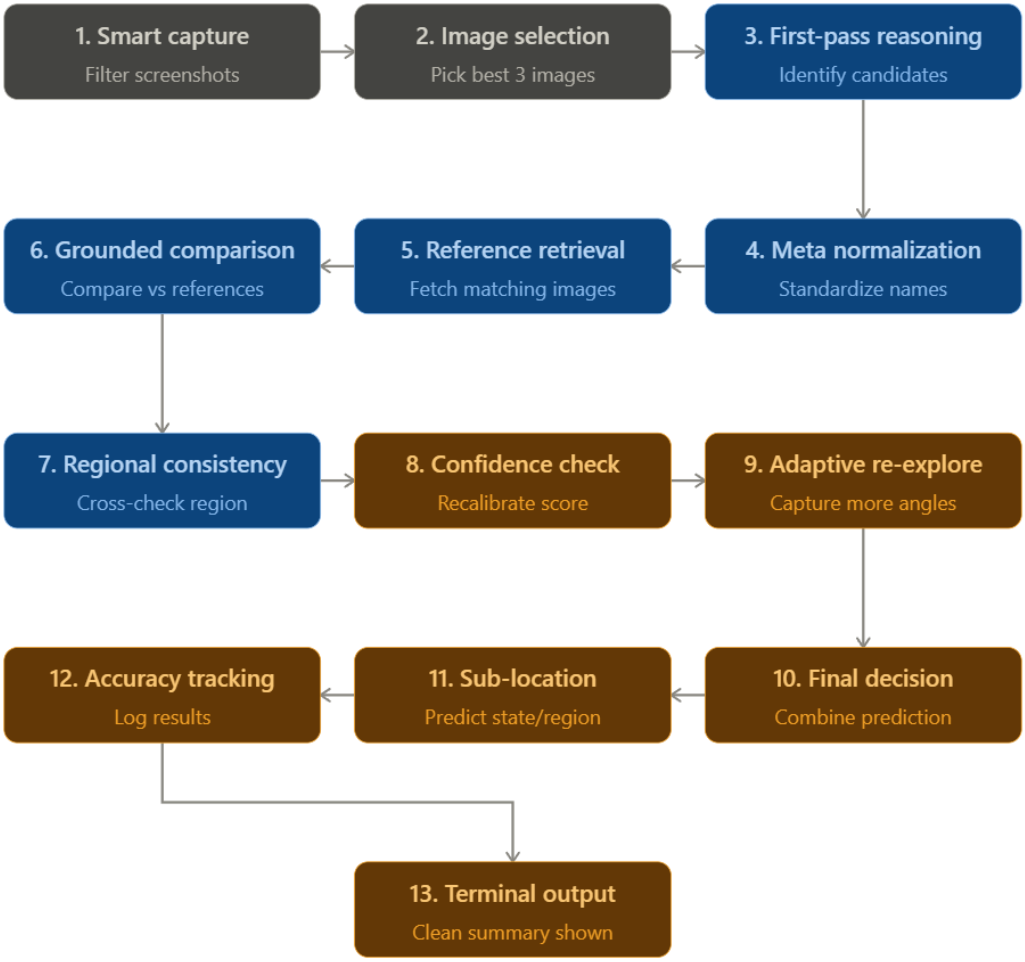
Ethical and Safety Considerations

1. **Physical Safety:** This is solely a software project with no physical hazards, no equipment risk and no chemical or biological materials involved.

2. **Data Ethics:** The reference image database was built by scraping geohints.com [8]. The reference image database is not included in the public GitHub repository because of copyright and terms-of-service considerations regarding redistributing content scraped from a third-party site. However, the scraper is included publicly thus, anyone can rebuild the database themselves using their own use of geohints.com.

3. **Dual-use risk and responsible handling:** Geolocation is inherently dual-use- the same capability which helps investigators verify footage or aid disaster response could, theoretically, be used to identify someone's location without consent, or support surveillance in ways which could potentially raise privacy concerns. This has been explicitly flagged in the 'Wider Purpose' section. Any kind of real-world deployment of this type of system would therefore, require careful consideration of consent and appropriate use policies. The Gemini API key is stored in a local .env file. This has also been excluded from the public repository.

Pipeline Implementation & Decisions



Grey: Input Processing

Blue: Core reasoning

Amber: Verification decision and output

Pinpoint consists of 13 stages; each has been implemented as a separate Python module. Every module performs one specific job and passes its output as a JSON file to the next stage. These JSON files are saved in an analysis folder in order to make the pipeline debuggable at every stage. This follows the design principle of modularity: stages can be troubleshot, replaced and improved without affecting the rest of the pipeline.

The 13 pipeline stages implemented are:

1. **Smart Capture (smart_capture.py)**
    - pyautogui captures screenshots of google street view images. Then, OpenCV scores each frame for various factors such as visual quality (sharpness and contrast) and geographic evidence density including presence of metas such as poles, bollards, signs and vegetation. This filtering is done because not every screenshot is equally useful. A blurry frame or repeated frame wastes time and cost therefore filtering before any API call is essential. The system keeps track of which directions have been captured and relaxes image quality thresholds for new directions to ensure that all angles are covered. Finally, the set of screenshots is sent down for processing in further stages.
2. **Image Selection (image_selector.py)**
    - After OpenCV filters the images, they are sent to Gemini to review and select the 3 best images based on which contain the most geographically useful visual information. Since sending all screenshots to every pipeline stage would be slow and expensive, a Gemini call is used to pick the best 3 images. This acts as a cost-effective filter.
3. **First-Pass Reasoning (country_reasoner.py)**
    - In this stage, another call is sent to Gemini to analyze all selected screenshots simultaneously across all 13 meta-categories and produce structured output- top candidate countries with confidence scores, a region cluster and evidence proof.
    - This is used to establish which countries are most plausible so that all subsequent stages operate only on these select candidates rather than considering all 125 countries.
4. **Meta Normalization (meta_normalizer.py)**
    - This stage standardizes the meta-category names output by Gemini into consistent internal names. For example, “vegetation” becomes “flora”, “utility_poles” becomes “poles”.
    - Since Gemini’s text output uses slightly different terms each run, without normalization, the retrieval engine would fail to find reference images because the category names will not match exactly.
5. **Reference Retrieval (retrieval_engine.py)**
    - In this phase, reference images are retrieved from the reference database for each candidate country from the first pass reasoning phase.

- This is done to enable grounded comparison- Instead of asking Gemini to rely on previous training knowledge, it gets reference examples of what each country's bollards, utility poles and road markings actually look like.
- Countries with more matching reference images across more categories are ranked higher. This is the evidence aware ranking system.

6. **Grounded Comparison (grounded_reasoner.py)**
   - Gemini receives the live screenshots along with the retrieved reference images side by side and produces match verdicts per meta category, per candidate country. This dramatically reduces the chance of confusing visually similar countries and significantly increases accuracy in the pipeline.
   - If Gemini returns a 429 rate-limit error, the system parses the retry delay from the error response, waits, and retries with the same reference images used previously.
   - To manage API costs and avoid these rate limit errors, the pipeline dynamically switches between Gemini 2.5 Flash and Gemini 2.5 Flash Lite based on usage. This ensures that the pipeline continues running even under heavy load.
7. **Regional Consistency (regional_consistency.py)**
   - This stage cross-checks whether the grounded reasoner's top country is geographically consistent with the region cluster which was established in the first pass reasoning phase.
   - This catches cases of pipeline drifts. For example, if the grounded reasoner picks Morocco but the first pass identified Central Asia as the strongest region, there must be an error somewhere as these are geographically incompatible.
8. **Confidence Verification (confidence_verifier.py)**
   - This stage recalibrates the confidence score based on strength of evidence. There are two special rules for this:
     - Definitive identifier floor: If a specific unambiguous identifier is detected, confidence is raised to at least 0.88 regardless of any other evidence.
     - Visually indistinguishable pairs: If a small enclave country wins (e.g. Lesotho) but its larger neighbor (South Africa) was within 0.15 confidence in the first pass and there is no floor specific identifier, the larger country is preferred. Examples of these pairs are: Eswatini/South Africa, Lesotho/South Africa, San Marino/Italy etc.
   - This is implemented because raw Gemini confidence is not always well calibrated. This improves system accuracy as it minimizes hallucination.
9. **Adaptive Re-exploration (adaptive_reexploration.py)**

- If the confidence level after verification is below a set threshold, the system triggers a new exploration cycle in street view to capture additional screenshots from different angles.
- Sometimes, the initial capture may miss the most informative view- a road sign or bollard, for example. A view from a different direction could drastically improve the country certainty

10. Final Decision (final_decision.py)

- This module combines the outputs from all previous stages and produces the final country prediction along with a confidence label (Strong/Moderate/Weak) and a recommendation (accept prediction/ seek signage / re explore)

11. Sub-Location Reasoning (sub_location_reasoner.py)

- For 15 major countries, when country confidence is above 0.75, the pipeline runs a separate Gemini call to narrow down to specific states or provinces using hand-crafted knowledge bases.
- The 15 countries are: USA, Australia, Brazil, India, Russia, Canada, Mexico, Argentina, Chile, Japan, South Africa, Indonesia, Nigeria, Kenya, Colombia
- Each country has a JSON knowledge base containing visual identifiers per region like state identifiers, route signs and Gemini reasoning hints.
- confidence calibration rules prevent overconfidence. If evidence applies to more than two states, confidence gets capped at 0.50; if 3+ states, capped at 0.35; if no distinguishing evidence, returns null rather than guessing.

12. Accuracy Tracking (accuracy_tracker.py)

- In order to track accuracy, when the final prediction is displayed, the user is prompted to enter the correct answer with a 30 second timeout. If provided, the round result is logged to accuracy_log/rounds.jsonl.
- Logged fields include: predicted country, correct country, whether correct, confidence label, decision source, failure stage classification, whether a definitive identifier was detected
- Running python main.py --stats produces a full accuracy summary- country accuracy %, most confused country pairs, failure stage breakdown, average confidence when correct vs incorrect

13. Terminal Output (terminal_output.py)

- This phase manages what gets displayed in the terminal. In order to prevent cluttering, a clean mode is set by default. One line is displayed per stage. A full verbose mode is also available in order to analyze detailed logs.

Problems Encountered & Solutions

1. **Region/Country Mismatch:**
   - **Problem**: In a run, the pipeline was predicting Morocco as the final country. However, the first-pass reasoning system identified the region cluster as central Asia. These are geographically incompatible. This was noticed during early testing runs.
   - **Cause**: regional_consistency.py was not enforcing that the final country must be geographically consistent with the strongest region cluster identified in the first-pass reasoning.
   - **Fix**: region enforcement was added in regional_consistency.py and a region guard was added in final_decision.py. This meant that the final country must now belong to the strongest regional cluster identified.
   - **Result**: geographically impossible predictions were completely eliminated.
2. **Movement timing:**
   - **Problem**: screenshots were initially being captured before street view finished loading after a movement or rotation. This produced blurry and unusable frames with near 0 geographical identifiers. This was noticed when the smart capture system was rejecting far too many images as low quality.
   - **Cause**: There was no wait time between movement commands and screenshot captures. This meant that the camera was still animating when the screenshot was captured.
   - **Fix**: Calibrated wait times were added- 1.8 seconds after movement, 1.2 seconds after rotation and 2.0 seconds after screenshot before proceeding.
   - Result: screenshot quality improved dramatically and far fewer frames were rejected.
3. **OCR adding significant time with zero value**:
   - **Problem**: When OCR was enabled and running on every frame, it added close to 1 minute per run. It also caused the pipeline to consider text present on the monitor screen such as date, time etc.
   - **Cause**: Gemini 2.5 Flash already reads and reasons about visible text in images directly. A separate phase for OCR was completely redundant.

- **Fix**: OCR was entirely disabled and its pipeline phase was completely removed.
- **Result**: Almost 1 minute was saved per run without any significant decrease in accuracy.

4. **False positive definitive identifiers:**
   - **Problem**: The confidence verifier system was incorrectly applying an extremely high confidence floor when it detected a US state highway code. However, it was actually misidentifying European road codes such as German L173 as US codes. This was noticed when runs in countries such as Germany and Australia were producing incorrectly high confidence scores pointing towards the USA.
   - **Cause**: The regex pattern for US state code detection was not strict enough. It matched any 2-letter prefix followed by numbers as a US highway code even though this road name format was used in many countries.
   - **Fix**: The regex was tightened to only match official 2 letter USPS postal codes and also added a country context check. Now, the definitive identifier rule only fires if the first-pass reasoning system also points towards the USA.
   - **Result**: European and Australian road codes were now being correctly identified. Their road codes now were logged as supporting evidence only rather than definitive identifiers.
5. **Smart capture rejecting new views as duplicates:**
   - **Problem**: After rotating the street view face to a new direction, the smart capture was flagging new screenshots as duplicates and rejecting them despite being new angles. This was identified when the pipeline was consistently missing road signs that were only visible from one direction.
   - **Cause**: The duplicate detection system threshold was too strict. A new direction with similar characteristic such as road type or vegetation was rejected despite being a new view.
   - **Fix**: Direction tracking was added. For any direction not yet captured, the threshold for duplicate detected was significantly lowered. This is also logged as a specific message so it can be monitored.
   - **Result**: directional coverage improved. Road signs and other directional clues were also reliably captured now.
6. **Rate limiting (429 errors):**
   - **Problem**: During runs containing many reference images, the pipeline was reaching Gemini's API rate limit and crashing. This was noticed during runs with a large number

of high-quality images leading to the pipeline crashing midway with no graceful recovery.

- **Cause**: There was no retry logic in the grounded reasoner. A 429 error caused the pipeline to fail.
- **Fix**: A proper 429 handling was added. The system now parses the retry delay from the error response, waits for a certain amount of time and retries with the same reference images and logs a warning. It may also dynamically switch to Gemini 2.5 Flash Lite depending on current usage limits.
- **Result**: The pipeline no longer crashes on rate limits.

7. **Sub-location overconfidence:**
    - **Problem**: A run returned "Western Australia" with 100% confidence based on karri forest vegetation and A440 route markings. However, the correct answer was Victoria. This was noticed during accuracy testing when sub-location predictions were being evaluated.
    - **Cause**: The same evidence existed across multiple Australian states. The sub-location reasoner did not account for the fact that some evidence was not state specific.
    - **Fix**: Confidence calibration rules were added. If evidence applies to 2 or more states, confidence is capped at 0.5; if 3 or more states, it's capped at 0.35. If there is no distinguishing evidence, the system returns a null value rather than a false positive.
    - **Result**: The sub-location overconfidence was dramatically reduced and the confidence level was much more representative of the pipeline's actual confidence.
8. **Eswatini/ South Africa enclave confusion**
    - **Problem:** In a run, Gemini's environmental analysis stated South Africa with 95% confidence but the final pipeline outputted Eswatini at 88% confidence. This was observed when comparing the pipeline's output against Gemini's own environmental analysis for the same location.
    - **Cause:** Eswatini is a small country bordering South Africa on the north, west, south, and southeast. It shares nearly identical visual identifiers such as left-hand driving, pine plantations and similar road markings. The grounded reasoning system scored Eswatini's reference images as a strong match because the countries look visually identical. Not because there was any definitive evidence pointing to Eswatini rather than South Africa.
    - **Fix:** A visually indistinguishable pairs system was added to the confidence verifier. If a small enclave country wins but its larger neighboring country was within 0.15 confidence in the first-pass reasoning and there was no enclave-specific evidence, the

larger country is preferred. Examples of these pairs are: Eswatini/South Africa, Lesotho/South Africa, Vatican City/Italy, San Marino/Italy.

- **Result:** Geographically tiny enclave countries no longer win incorrectly over larger countries if there is no enclave-specific evidence.

Results & Accuracy

1. **Country-level accuracy:**

Metric	Result
Total rounds tested	50
Correct country predictions	41
Country accuracy	82%
Rounds with definitive identifier	23
Accuracy when definitive identifier detected	100%

2. **State-level accuracy:**

Metric	Result
Countries with state-level reasoning	15
State-level accuracy	~76%
Rounds where state attempted	25
Rounds where state/region correct	19

3. **Comparison against single-model prompting:** To evaluate whether the multi-stage pipeline meaningfully improves on single-model prompting, 25 rounds were also tested using a single direct Gemini call with no reference database, retrieval, or confidence calibration

Metric	Single-Gemini Call	Pinpoint (Full Pipeline)
Rounds Tested	25	25 (same locations)
Correct Country Predictions	14	21
Country Accuracy	56%	84%
Average Confidence	~95%	~85%

Strikingly, the single-model prompt demonstrates an average confidence of 95% despite achieving only 56% accuracy. This means that it was frequently wrong while reporting with high levels of confidence i.e. it was hallucinating. On the other hand, Pinpoint's average confidence

closely matched its actual accuracy (85% vs 84%). This shows that the confidence calibration system outputs scores that meaningfully reflect the probability of a correct answer.

Conclusions and Wider Implications

Results:

- The 82% country accuracy shows that Pinpoint's multi-stage pipeline with a grounded reference comparison drastically outperforms single-model prompting for geographic identification.
- The ~100% accuracy on rounds with definitive identifiers confirms the confidence calibration logic. High confidence is assigned to runs when strong confidence exists.
- Most of the errors occur in two cases:
    - **Visually similar countries**: This includes countries such as France and Monaco where there are not enough visual identifiers to distinguish between the two accurately.
    - **Insufficient capture**: This happens when the capture system is unable to capture visually significant evidence, due to factors such as lack of metas in the location or bad image quality.

What was set out to be done vs what was achieved

- **Objective 1 (research metas**): Achieved 13 meta-categories. These were defined and validated through geohints.com
- **Objective 2 (reference database)**: built a reference database with 13 meta categories and 1840+ images across 125 countries.
- **Objective 3 (multi-stage pipeline):** built a pipeline with 13 stages. Each stage has its own unique purpose and is individually implemented.
- **Objective 4 (state-level reasoning):** This was achieved for 15 countries, exceeding the original target expectation of 10.
- **Objective 5 (accuracy tracking):** accuracy_tracker.py was implemented along with a --stats flag

- **Objective 6 (identify and fix problems and failures):** identified, documented and fixed the root cause of 8 bugs.

Wider Implications

- **OSINT:** the 82% accuracy shows that geographic reasoning can be automated meaningfully. For OSINT purposes, an 82% accuracy system which is able to flag likely locations for human verification can significantly reduce analyst workload.
- **Border security/law enforcement:** Like OSINT, a system which is able to narrow down location to a likely region or country without GPS metadata has genuine utility as a first-pass filter system for analysts.
- **Aerial/ Satellite imagery:** the meta-category framework and approach- identifying and reasoning about visual patterns and identifiers consistent within geographic regions- is not limited to street-level imagery. This approach can also be applied to aerial and satellite imagery for disaster response and monitoring land use.
- **Pipeline architecture:** Pinpoint also demonstrates that a single prompt model is insufficient for complex visual reasoning tasks. A multi stage pipeline produces significantly better results. Apart from visual reasoning, this finding can also be generalized to domains beyond geography.
- Pinpoint is a proof of concept tested on Google street view and GeoGuessr imagery. Real world application and deployment would require significantly more testing, a larger reference database and domain specific adaptation.

Reflections and Future Work

What went well:

- The reference database approach was definitely the best pipeline architectural decision. Having grounded comparison against real examples was much more reliable than using pure model reasoning, which relies solely on previous training.
- The modular pipeline design was extremely helpful as bugs and problems could be found and troubleshot without breaking the rest of the system. This helped me in fixing problems such as the South Africa/Eswatini case where isolating the failing stage would have been extremely difficult using a single API call.

- While trying to populate the reference database, initially things did not go as expected as none of the images were stored on the geohints.com website directly. This made building the database a genuine challenge, but I felt a strong sense of satisfaction after achieving this feat.
- The confidence calibration work, which included enclave pairs and sub-location caps, encouraged me to think more carefully about edge cases. This separated pinpoint from a demo to something which I felt was closer to a working quality project.

Things which did not go as planned:

- OCR (Optical Character Recognition): Initially, I was working on implementing OCR into the pipeline using OpenCV in hopes that it would increase the pipeline's accuracy by extracting text from images. However, I soon realized that this was not required as Gemini could extract text from images itself. Also, this increased the time for each run without providing any significant benefit.
- Sub location accuracy: It is slightly lower than I expected. This is mainly due to 2 reasons. First, in many countries, a lot of visual identifiers are shared between multiple states. Many times, the evidence is not distinctive enough for the model to be certain. Second, since I had to mainly create the state references by hand, there was not sufficient data for extremely accurate state level identification.
- Speed: The pipeline is slower than I expected. When I first got the idea, I expected it to be extremely fast. I thought that it could identify countries in less than 1 minute. However, after completing it I realized it was taking around 2 minutes per round. This mainly occurs because of using slower AI models. Also, finding visual matches through a reference system takes a while.
- The systematic accuracy tracking was one of the last phases I implemented. Because of this, I had to judge early pipeline changes informally rather than against concrete data.

What I learned:

- The primary skill which I developed through this project was how to design a multi-stage AI pipeline. I learned how to break a complex reasoning task into separate phases and documentable steps.
- Before making this, I was not very familiar with prompt engineering. To make detailed prompts, I would use ChatGPT GPTs like Prompt Professor. However, after learning

how to specifically phrase prompts to my liking, I was able to dramatically change the structure and reliability of Gemini prompts.

- While making the pipeline, since each phase was logged and displayed in the terminal, I was always able to see exactly what was going wrong in the pipeline. I fixed every bug by first inspecting JSON outputs, finding the root problem and then approaching the problem through careful research and consideration.
- I also learned the importance of calibration. A system which knows when it might be wrong or uncertain is much more useful than something which constantly hallucinates or is constantly wrong. Although building this into the pipeline sometimes made producing confident outputs slightly challenging, it helped significantly in the long run.

What I would do differently and future developments:

- If I was starting again, I would make a few changes:
    - I would definitely build the accuracy tracker earlier. Without this, gathering early changes quantitatively was a big challenge as I had to rely on gut feeling and memory rather than actual data.
    - I would also consider spending more time on the first-pass reasoning system as it is the highest leverage stage in the entire pipeline.
- **Future development directions:**
    - I will work towards adding a city level or coordinate level output system to make the results more and more accurate. When starting, I would primarily target cities which have very distinct geographical identifiers such as a special type of architectures.
    - I will also reduce the pipeline execution time and expand the state level reasoning beyond the 15 countries that are currently implemented.
    - Apart from this I will be working on growing the reference database to add more meta categories and countries for better global coverage and country recognition.
    - After working on all of this, I will approach companies working in fields where Pinpoint would be helpful to field test the pipeline and evaluate whether it has significance and use in these industries.

References

- [1] Haas, L., Skreta, M., Alberti, S. & Finn, C. (2023). PIGEON: Predicting Image Geolocations. arXiv:2307.05845. Presented at CVPR 2024. https://arxiv.org/abs/2307.05845
- [2] Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems, 33, 9459–9474. https://arxiv.org/abs/2005.11401
- [3] Wei, J., Wang, X., Schuurmans, D., et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. Advances in Neural Information Processing Systems. https://arxiv.org/abs/2201.11903
- [4] Huang, L., Yu, W., Ma, W., et al. (2023). A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. arXiv:2311.05232. https://arxiv.org/abs/2311.05232
- [5] Chen, J., Lin, H., Han, X. & Sun, L. (2024). Benchmarking Large Language Models in Retrieval-Augmented Generation. Proceedings of the AAAI Conference on Artificial Intelligence. https://arxiv.org/abs/2309.01431
- [6] Bellingcat (2015). Searching the Earth: Essential Geolocation Tools for Verification. https://www.bellingcat.com/resources/how-tos/2015/07/25/searching-the-earth-essential-geolocation-tools-for-verification/
- [7] Google DeepMind. *Gemini API Documentation.* Google.
    - Available at: https://ai.google.dev/gemini-api/docs
- *[8] geohints.com — GeoGuessr Meta Reference Database.*
    - Available at: https://geohints.com
- [9] OpenCV
    - *OpenCV Documentation a*vailable at: https://opencv.org
- [10] pyautogui
    - *pyautogui Documentation a*vailable at: https://pyautogui.readthedocs.io
- [11] Python Software Foundation. *Python 3 Documentation*
    - Available at: https://python.org
- [12] Anthropic. *Claude Documentation.*
    - Available at: https://docs.claude.com (used to reference Claude AI and Claude Code, declared under Materials & Resources / AI Assistance)

- **[13]** *GeoGuessr.* Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/wiki/GeoGuessr — designer Anton Wallén, released May 2013.